Davante

12

PROGRAMACIÓN

Flujos de información

ÍNDICE

/ 1. Introducción y contextualización práctica 4

/ 2. Stream 5

/ 3. Tipos de flujos 5

/ 4. Caso práctico 1: “Almacenando datos” 6

/ 5. Flujos predeterminados 7

/ 6. Flujos basados en bytes 8

/ 7. Flujos basados en caracteres 9

/ 8. Ficheros 10

/ 9. Clasificación de los ficheros 11

9.1. Clasificación de acuerdo al tipo de acceso: Secuencial 11

9.2. Clasificación de acuerdo al tipo de acceso: Aleatorio 12

9.3. Clasificación de acuerdo al tipo de organización 13

9.4. Clasificación de acuerdo al formato 13

/ 10. Caso práctico 2: “Lectura de caracteres” 14

© MEDAC ISBN: 978-84-19885-69-2
Reservados todos los derechos. Queda rigurosamente prohibida, sin la autorización escrita de los titulares del copyright, bajo las sanciones establecidas en las leyes, la reproducción, transmisión y distribución total o parcial de esta obra por cualquier medio o procedimiento, incluidos la reprografía y el tratamiento informático.

ÍNDICE

/ 11. Ruta de los ficheros 14

/ 12. Errores de ficheros 15

/ 13. Resumen y resolución del caso práctico de la unidad 16

/ 14. Bibliografía 18

OBJETIVOS

Comprender el concepto de flujo.

Conocer los diferentes flujos existentes.

Entender el concepto de fichero y sus distintas clasificaciones junto a las diferentes formas de acceso a los mismos.

Conocer las jerarquías de flujos y sus direcciones.

Dominar las diferentes excepciones asociadas a flujos.

/ 1. Introducción y contextualización práctica

En esta unidad, vamos a tratar los conceptos de flujo y fichero. Vamos a ver qué es un flujo, para qué sirve y los diferentes tipos de flujos que nos podemos encontrar, que darán lugar a los ficheros que todos conocemos.



Fig.1. Fichero.

También aprenderemos las diferentes formas de acceso a ficheros que existen, con sus ventajas e inconvenientes correspondientes y las distintas formas de referenciar a los ficheros, lo que se conoce como rutas.

Por último, analizaremos el tratamiento de ficheros, donde observaremos que este es susceptible a provocar errores, y estudiaremos cuáles son estos.

A continuación, vamos a plantear un caso práctico a través del cual podremos aproximarnos de forma práctica a la teoría de este tema.

Escucha el siguiente audio donde planteamos la contextualización práctica de este tema. Encontrarás su resolución en el apartado 'Resumen y resolución del caso práctico'.

Audio intro. "Persistencia de datos"

https://on.soundcloud.com/9dEjhXFFfrXg5Gpu7

/ 2. Stream

Hasta el momento, nuestros programas siempre han funcionado de la misma forma: solicitando información al usuario para que la introduzca por teclado, procesando dicha información y mostrando por pantalla el resultado del procesamiento.

Una vez que nuestros programas se han ejecutado y cerrado, toda esa información que se ha introducido se pierde, ya que se almacenaba en la memoria RAM del ordenador y el programa, al cerrarse, se desaloja de memoria, borrando toda información relativa al mismo.

Pero ¿qué ocurre si queremos que esa información introducida no se pierda al cerrar el programa? Puede ser que la necesitemos para futuras ejecuciones. Es ahora donde entran en juego los flujos a ficheros. Un flujo de datos es una vía de comunicación entre dos puntos, un origen y un destino. Una buena analogía a un flujo de datos es un sistema de **tuberías** con entradas (sumideros) y salidas (grifos) de agua (información), o una **carretera** (como muestra la figura), donde circulan vehículos, que, como podemos imaginar, son la información que se transmite a través del flujo.

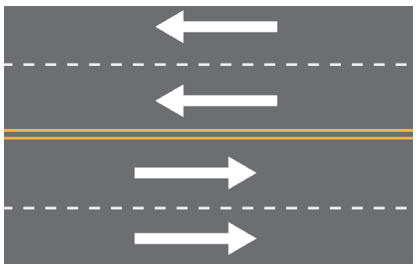


Fig.2. Símil de flujo de información y carretera.

Esta información transmitida podrá ir en dos direcciones, siendo estás los flujos de entrada y de salida de información. Sin embargo, no pueden ir en ambas direcciones al mismo tiempo, es decir, son unidireccionales y se utilizará solo para leer o escribir.

Mediante el uso de flujo podremos crear ficheros en disco, pudiendo almacenar y recuperar información de ellos cada vez que lo necesitemos.

Los flujos nos van a aportar una gran ventaja, que es la persistencia de datos. Con esto nos referimos a que se va a poder guardar información en disco, y gracias a esto, cuando cerremos el programa, la información no se va a perder, siempre y cuando la hayamos almacenado en un fichero antes de cerrar, permitiéndonos así recuperarla cuando lo necesitemos para volver a operar con ella.

Los flujos a ficheros también tienen un gran inconveniente, y es que el acceso a disco, tanto para almacenar como para recuperar información, es muy ineficiente en cuanto a tiempo se refiere, es decir, se tarda mucho en almacenar y recuperar información de un fichero.

Lo más recomendable es realizar los mínimos accesos, para paliar la ineficiencia del acceso.

/ 3. Tipos de flujos

Cualquier lenguaje de programación actual provee el uso de flujos de datos para poder operar y utilizar la persistencia de datos en sus programas.

En nuestro caso, el lenguaje de programación Java nos provee de una serie de clases (se encuentran en el paquete java.io.) que forman una jerarquía para poder tener acceso a ficheros e implementar la persistencia de datos. Esta jerarquía se va a dividir en dos partes:

- **Clases para lectura de ficheros**.
- **Clases para escritura de ficheros**.

Se podría decir, por tanto, que (en Java) el flujo de datos se refiere al movimiento de datos entre diferentes fuentes y destinos. Hay varios tipos de flujos en Java que se utilizan para leer y escribir datos desde y hacia diferentes fuentes, como archivos, dispositivos de entrada/salida, redes, etc.

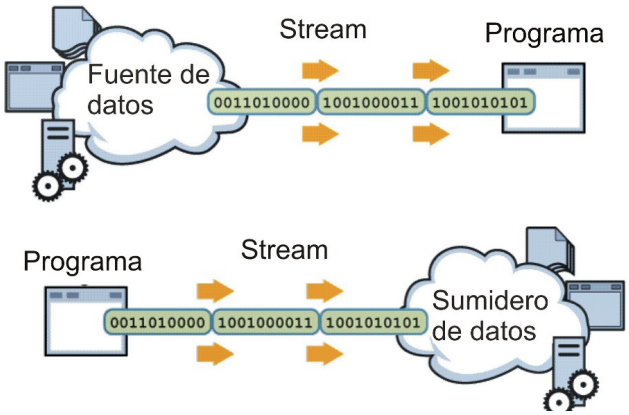


Fig.3. Flujo de datos.

Algunos tipos comunes de flujos en Java:

- Flujos de salida.
- Flujos de caracteres.
- Flujos de bytes.
- Flujos de objetos.
- Flujos de datos.
- Flujos de entrada de lectura aleatoria.

La **ventaja** de utilizar flujos de datos, es que estos van a ser independientes del tipo de dispositivo que utilicemos, es decir, que va a ser indiferente si estamos trabajando con un flujo que obtenga información de teclado, o un flujo que recoja información de un fichero en una determinada ruta. Si lo hemos implementado de forma correcta, no debería haber problema en ello, habiendo liberado al programador que lo implementa de saber con qué está actuando.

Esta vinculación del sistema físico con el tipo de flujo la hará la propia máquina virtual de Java, es decir, el JDK. Resumiendo, quien tendrá que encargarse de comunicarse con el sistema operativo va a ser el flujo, de esta manera, no vamos a tener que cambiar mucho en nuestra aplicación para hacer que un programa que originalmente obtenía información de teclado ahora la obtenga de un fichero de datos almacenado en disco. Esta gran capacidad es una característica vital para un lenguaje de programación multiplataforma como es Java.

/ 4. Caso práctico 1: “Almacenando datos”

Planteamiento: La escritura de texto en ficheros en Java es esencial para la persistencia de datos, permitiendo el almacenamiento y la recuperación de información más allá del ciclo de vida de la aplicación.

Nudo: Se acaba de recibir un nuevo encargo para desarrollar un nuevo sistema que permita almacenar de forma persistente los datos de una serie de empleados de un taller mecánico. Para una primera fase del desarrollo, se ha encargado el diseño en pseudocódigo de las acciones necesarias para almacenar los empleados en el menor tiempo posible.

Desenlace: La forma más sencilla y rápida de llevar a cabo el almacenamiento de datos en un fichero que nos permita tener un almacenamiento persistente es ir recorriendo dichos datos (los empleados del taller, en este caso) uno a uno, e ir almacenándolos a la vez que se recorren.

Una cosa que se debe tener en cuenta es que toda operación con ficheros podrá lanzar una serie de excepciones que tenemos que el IDE nos pedirá controlar y completará mediante un bloque try-catch. Aunque es un concepto que aún no se ha visto, se tratará pronto al ser de vital importancia, ya que si no lo hacemos, el programa podrá generar un fallo en tiempo de ejecución y, además de cerrarse, puede perderse toda la información que se estaba almacenando, ya que la que no se escribiera se habrá perdido, y la que se pudiera haber escrito puede perderse porque el fichero de guardado puede corromperse al no cerrarse de forma adecuada.

Un ejemplo del pseudocódigo que se ha solicitado podría ser el mostrado en la siguiente imagen:

```
// Abrimos el fichero de los datos
fichero = abrirFichero()
// Recorremos un array con los datos
desde 0 hasta cantidad_datos
    // Obtenemos el dato a guardar
    empleado = obtenerEmpleado()
    // Guardamos los datos del empleado
    escribirEnFichero(fichero, empleado)
// Una vez terminemos cerramos el fichero
cerrarFichero(fichero)
```

Fig.4. Pseudocódigo de almacenamiento de datos.

/ 5. Flujos predeterminados

Todos los lenguajes de programación que han existido y existen se han provisto de flujos para poder introducir y mostrar información, lo que se conoce como los flujos de entrada y salida estándar. Esto permite que se pudieran leer datos de teclado y mostrar datos por pantalla.

Ya sabemos qué son los flujos y que hay principalmente dos tipos: los de entrada que permiten la posibilidad de introducir datos hacia un programa y los de salida de información encargados de mostrar información al usuario.

Audio 1. "Entrada y salida de información"
https://bit.ly/32XPm3q

Hasta el momento, (tal y como muestra la figura) hemos estado utilizando **flujos de entrada y de salida** sin ser conscientes de ellos, los cuales son **los estándares**, tanto para la entrada como para la salida:



Fig.5. E/S estándar de Java.

Veamos algunos de ellos:

- **System.in**: Este es un flujo de entrada estándar que representa la entrada del teclado. Se utiliza con la clase Scanner (como muestra la figura) o BufferedReader para leer datos del teclado.

```java
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Ingrese un número: ");
        int num = scanner.nextInt();
        System.out.println("Número ingresado: " + num);
        scanner.close();
    }
}
```

Fig.6. Flujo de entrada.

- **System.out**: Este es un flujo de salida estándar que representa la salida a la consola. Se utiliza con System.out. println() para imprimir datos en la consola.

- **System.err**: Existe un flujo de salida estándar concreto para errores que normalmente se usa para imprimir mensajes de error (como muestra la figura) en la consola. Su resultado es exactamente igual que la salida normal (System.out), salvo que las letras aparecerán en color rojo.

```java
public class Main {
    public static void main(String[] args) {
        System.err.println("¡Ha ocurrido un error!");
    }
}
```

Fig.7. Flujo de error.

¡! Sabías que...

Los flujos System.out y System.err son bufferizados, lo que significa que los datos pueden no aparecer inmediatamente en la consola. Puedes forzar el vaciado del buffer usando System.out.flush() o System.err. flush().

/ 6. Flujos basados en bytes

Los flujos basados en bytes están orientados tanto a la lectura como a la escritura de información organizada en bytes.

Este tipo de flujos, por su configuración y funcionamiento, son ideales para gestionar ficheros que contengan información estructurada de forma binaria.

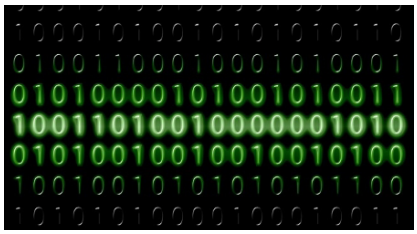


Fig.8. Lectura y escritura de bytes.

El lenguaje de programación Java provee dos clases que nos van a proporcionar toda la funcionalidad necesaria para poder tratar con este tipo de flujos:

- InputStream.
- OutputStream.

Con la clase **InputStream**, podremos crear flujos de entrada de datos a ficheros basados en bytes, es decir, podremos **leer** información de ellos.

> **¡! Sabías que...**
>
> System.in es un InputStream, lo que significa que proporciona bytes de datos. Para leer caracteres o líneas de texto, es común envolver System.in en un InputStreamReader y un BufferedReader.

Con la clase **OutputStream**, podremos crear flujos de salida de datos a ficheros basados en bytes, es decir, podremos **escribir** información en ellos.

A los objetos instanciados de estas clases podremos indicarles dónde se encuentra el fichero al que queramos abrir el flujo, es decir, la ruta del fichero para que puedan acceder a él.

Del mismo modo que podemos indicar la ruta del fichero al que queremos abrir un flujo, podremos indicarles también los tipos predeterminados, por lo que tendremos lectura y escritura 'normal'.

Las operaciones con este tipo de flujos siempre van a constar de cuatro partes:

- Abrir el flujo al fichero.
- Comprobar que el flujo se ha abierto correctamente.
- Operar con el fichero.
- Cerrar el fichero.



Vídeo 1. "Jerarquía de clases de flujo 1"
https://bit.ly/355LNKY



/ 7. Flujos basados en caracteres

Con los flujos orientados a bytes podemos tener la funcionalidad necesaria para poder trabajar con la información de una forma muy sencilla, pero estos tienen una limitación importante: no pueden trabajar con caracteres Unicode, ya que estos utilizan 2 bytes para su codificación.

En principio, esta consideración puede parecer no tener demasiada importancia, pero debido a esto, se consideró crear un nuevo tipo de flujo de datos: el flujo basado en caracteres.

Este nuevo tipo de flujo de datos ofrece la funcionalidad necesaria para poder **trabajar directamente con caracteres**, pudiendo operar con **Unicode y todas sus respectivas variaciones**.

Dependiendo de la funcionalidad que necesitemos, existen muchos tipos de flujos de datos. Para la implementación de los flujos de caracteres, Java nos va a proporcionar dos clases base:

- Reader.
- Writer.

Estas dos clases pueden tratar estos flujos, de carácter en carácter, lo cual hace que su tratamiento sea muy lento, y más teniendo en cuenta la ineficiencia del acceso a disco.

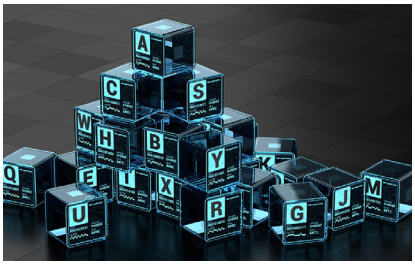


Fig.9. Estándar universal de codificación de caracteres.

Para solucionar esto, lo que haremos será crear otros flujos a partir de estos dos. Estos nuevos flujos van a ser:

- **InputStreamReader**: Podremos crear flujos de entrada de datos a ficheros basados en caracteres, es decir, podremos leer información de ellos.
- **OutputStreamWriter**: Podremos crear flujos de salida de datos a ficheros basados en caracteres, es decir, podremos escribir información en ellos.

Al igual que con los flujos anteriores, las operaciones con este tipo de flujos siempre van a constar de cuatro partes:

- Abrir el flujo al fichero.
- Comprobar que el flujo se ha abierto correctamente.
- Operar con el fichero.
- Cerrar el fichero.

Vídeo 2. "Jerarquía de clases de flujo 2"
https://bit.ly/2Z5J8gu

/ 8. Ficheros

Hasta ahora hemos introducido información de entrada al sistema y generado una salida al sistema. Sin embargo, para manejar grandes cantidades de información, es necesario emplear el uso de archivos o ficheros.

Un fichero es un **conjunto de registros donde cada registro está compuesto de una serie de campos**. Un registro es una estructura de datos ya definida que representa un fichero, y mediante esta representación, podemos saber perfectamente dónde se encuentran todos sus elementos, como pueden ser los metadatos, la parte de información, los detalles, etc.

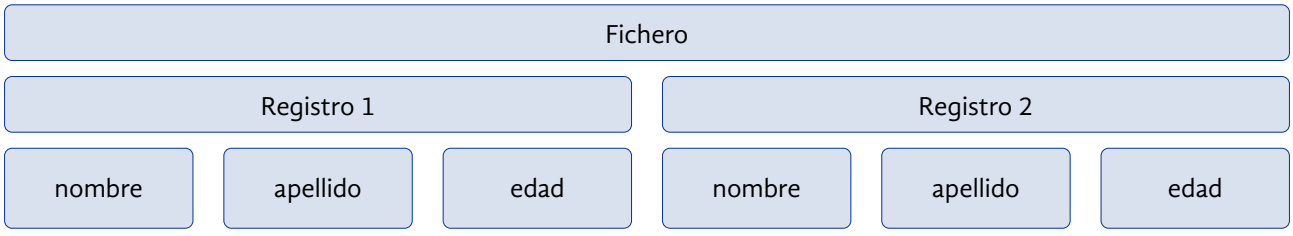


Fig.10. Estructura de un fichero.

En Java esto no ocurre, por lo que al no haber una estructura de datos ya definida para indicar cómo son los ficheros, son los propios programadores los que tienen que decidir cómo se van a estructurar estos.

Esto tiene tanto ventajas como inconvenientes. Por ejemplo, proporciona un nivel de personalización muy alto, pero, por el contrario, hace que tengamos que explicar cómo están estructurados los ficheros de nuestros programas para que otros los puedan utilizar.

Con respecto a los registros, debemos tener claro los siguientes conceptos:

- **Registro lógico**: Conjunto de información de uno de los elementos que hace referencia al fichero. El creador del fichero determina las propiedades de cada registro y en consecuencia su tamaño.
- **Registro físico o bloque**: Un registro físico es el conjunto de información que puede ser escrito o leído de una sola vez. Esta información no tiene por qué corresponderse con un registro lógico y, por norma general, se podrán escribir o leer varios registros de una vez.
- **Factor de bloque**: Es el número de registros contenidos en un bloque Este es muy importante en el diseño de los mismos, ya que influirá de forma directa en la velocidad de lectura/escritura.
- La **dirección lógica** es la dirección relativa dentro del fichero que ocupa un bloque, mientras que la **dirección física** es la dirección real que ocupa dicho bloque en el sistema de almacenamiento que se encuentre.

Los archivos suelen tener un campo clave para los procesos de búsqueda para identificar de manera unívoca cada registro.

/ 9. Clasificación de los ficheros

Los ficheros pueden tener distintas clasificaciones en base al soporte físico que lo implementa, es decir, donde se guardan los datos. Estos tipos pueden ser:

- Soportes secuenciales.
- Soportes direccionable.

Los **soportes secuenciales** requieren acceso en orden secuencial, mientras que los **direccionables** permiten acceso directo a posiciones específicas mediante direcciones.

9.1. Clasificación de acuerdo al tipo de acceso: Secuencial

En este tipo de ficheros, los registros que almacenan la información se almacenan de forma secuencial o consecutiva, es decir, un dato detrás de otro. La gran consecuencia de este tipo es que cualquier operación que necesitemos hacer sobre uno de estos ficheros se hará sobre el mismo orden lógico en el que fueron insertados sus registros.

Con estos ficheros, vamos a poder realizar las siguientes operaciones:

- **Añadir elementos**: Cuando vayamos a añadir un nuevo elemento en este tipo de ficheros, solo lo podremos hacer al final del mismo, justo después del último registro que tengamos.
- **Eliminar elementos**: Para poder eliminar un registro en este tipo de ficheros, primero tendremos que localizarlo, y una vez localizado, se podrá suprimir, teniendo que mover los registros de delante una posición atrás para seguir con la estructura lógica.

- **Modificar elementos**: Para poder modificar un elemento, primero tendremos que localizarlo y luego podremos modificar su valor.

- **Consultar elementos**: Para poder consultar un elemento, tendremos que hacerlo en orden secuencial, es decir, habrá que recorrer todos los registros anteriores al registro que queremos localizar.

Este tipo de ficheros son buenos para operaciones de programas que no sean interactivos, ya que tendrán pocos accesos a ellos y los que tengan serán fáciles de implementar. Tienen la ventaja de que aprovechan bien el espacio en memoria reservado para ellos.

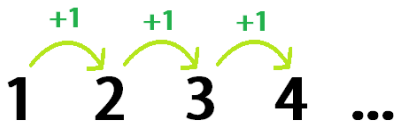


Fig.11. Elementos ordenados.

9.2. Clasificación de acuerdo al tipo de acceso: Aleatorio

Los ficheros de acceso aleatorio surgieron como **una mejora** de los de acceso secuencial.

Este tipo de ficheros son mucho **más versátiles** que los de acceso secuencial y **permiten acceder a cualquier parte del fichero, en cualquier momento y de forma directa**, como si se tratase de un acceso a memoria.

La **flexibilidad** que nos otorga esta propiedad es muy alta, ya que no tendremos que acceder a todos los registros anteriores al registro al que queremos acceder, ahorrando una gran cantidad de tiempo.

Con estos ficheros, vamos a poder realizar las siguientes **operaciones**:

- **Añadir elementos**: Cuando vayamos a añadir un nuevo elemento en este tipo de ficheros, lo podremos hacer en cualquier punto del mismo, basta con indicar dónde lo queremos añadir, teniendo que mover los registros de delante una posición hacia adelante para seguir con la estructura lógica.

- **Eliminar elementos**: Al igual que con el acceso secuencial, para poder eliminar un registro en este tipo de ficheros, primero tendremos que localizarlo, y una vez localizado, se podrá suprimir, teniendo que mover los registros de delante una posición atrás para seguir con la estructura lógica.

- **Modificar elementos**: La modificación también comparte el mismo proceso que en el acceso secuencial, es decir, para poder modificar un elemento, primero tendremos que localizarlo y, luego, modificarlo.

- **Consultar elementos**: Para poder consultar un elemento, lo podemos hacer directamente, indicando la posición del registro.

- Cuando creamos un flujo a este tipo de ficheros, tendremos que indicar qué tipo de operación queremos realizar con ellos, si la escritura o la lectura.

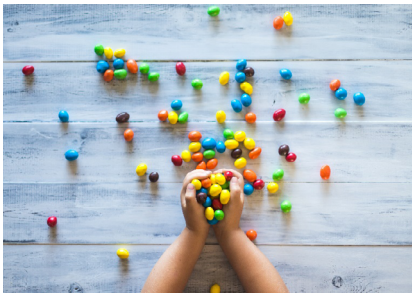


Fig.12. Elementos ordenados.

A modo **resumen**, entre estos dos tipos de acceso, podría decirse que el acceso secuencial implica recorrer registros en orden consecutivo, mientras que el acceso directo permite acceder a cualquier registro sin necesidad de recorrer los anteriores. Esta clasificación no se corresponde directamente con el tipo de soporte utilizado. El acceso secuencial puede implementarse en soportes secuenciales o direccionables, mientras que el acceso directo solo es posible en soportes direccionales.

9.3. Clasificación de acuerdo al tipo de organización

En cuanto a la forma en la que se han guardado los datos dentro del fichero, existen **tres tipos**:

- **Secuencial**: La forma de acceder a los datos es reconociéndolos en orden hasta un último registro llamado EOF (*end of file*). Este tipo de ficheros se puede implementar tanto en soporte aleatorio como secuencial.

- **Organización directa o aleatoria**: El orden físico no se corresponde con el orden lógico. Nos permite leer o escribir donde queramos. Hace falta algoritmos para calcular el orden lógico. Solo es implementable sobre soportes de tipo direccionables o aleatorios.

- **Organización secuencial indexada**: Tal y como muestra la figura, emplea dos archivos: el primer archivo es el índice o archivo de claves; el segundo contiene la información. El archivo de índices es secuencial y el de información es aleatorio. Solo se implementa sobre soportes direccionables.



Fig.13. Organización secuencial indexada.

9.4. Clasificación de acuerdo al formato

En función del formato empleado para almacenar la información, existen distintos **tipos**:

- **Ficheros binarios**: Aquellos ficheros cuyo contenido son secuencias de dígitos binarios que solo pueden ser leídos por un programa en lugar de un editor de textos.

- **Ficheros de texto**: Aquellos ficheros cuyo contenido es una secuencia de caracteres y puede, por tanto, ser leído por un editor de textos.

- **Ficheros XML**: Estos son ficheros que contienen datos estructurados en formato XML (*Extensible Markup Language*). XML es un lenguaje de marcado que permite representar datos de manera jerárquica y legible. Se utilizan bibliotecas como DOM, SAX, JAXB o StAX para leer y procesar ficheros XML en Java.

- **Ficheros que contienen datos estructurados en formato JSON (*JavaScript Object Notation*)**: JSON es un formato de intercambio de datos ligero y fácil de leer, que se utiliza comúnmente en aplicaciones web y servicios web. Se emplean bibliotecas como Gson o Jackson para leer y procesar ficheros JSON en Java.

Estos son algunos de los tipos de ficheros de lectura comunes en Java. La elección del tipo de fichero dependerá de los requisitos específicos de nuestra aplicación y del formato en el que estén almacenados los datos que deseamos leer.

/ 10. Caso práctico 2: "Lectura de caracteres"

Planteamiento: Los flujos carácter a carácter en Java, representados por las clases Reader y Writer, permiten la lectura y escritura de datos en forma de caracteres Unicode, facilitando el manejo de texto. Esto implica que tanto la lectura como la escritura se hagan carácter a carácter mediante clases como FileReader y FileWriter hasta el final del archivo. Estas operaciones pueden mejorar su eficiencia mediante el uso de un buffer.

Nudo: Se pide realizar el pseudocódigo de una función que permita abrir un fichero de datos, realizar la lectura carácter a carácter del fichero e imprimir la cadena de texto leída.

Desenlace: La lectura de caracteres se realiza de manera general en un bucle que continúa hasta que se encuentra el valor -1, que indica el final del flujo. Esto se hace usando métodos como read() de las clases Reader o InputStreamReader.

Cada llamada a read() devuelve el siguiente carácter como un entero, o -1 si se ha alcanzado el final del archivo o flujo de entrada. Este enfoque permite procesar eficientemente el texto, ya que se leen los datos carácter por carácter hasta que no hay más que leer, asegurando que se maneja todo el contenido correctamente y se detecta el fin del flujo de manera adecuada.

Un ejemplo de la función solicitada podría ser la mostrada en la siguiente imagen:

```
Abrimos el fichero de los datos
Lectura de un caracter del fichero abierto
Mientras caracterLeido no sea fin de fichero
    Concatena el caracter leido a la cadena de texto
    Lectura de un caracter del fichero abierto
Fin Mientras
Imprimir cadena de texto
```

Fig.14. Lectura de un fichero carácter a carácter.

/ 11. Ruta de los ficheros

Los ficheros se van a almacenar en disco dentro de la jerarquía de carpetas en la que dividamos nuestro sistema de archivos.

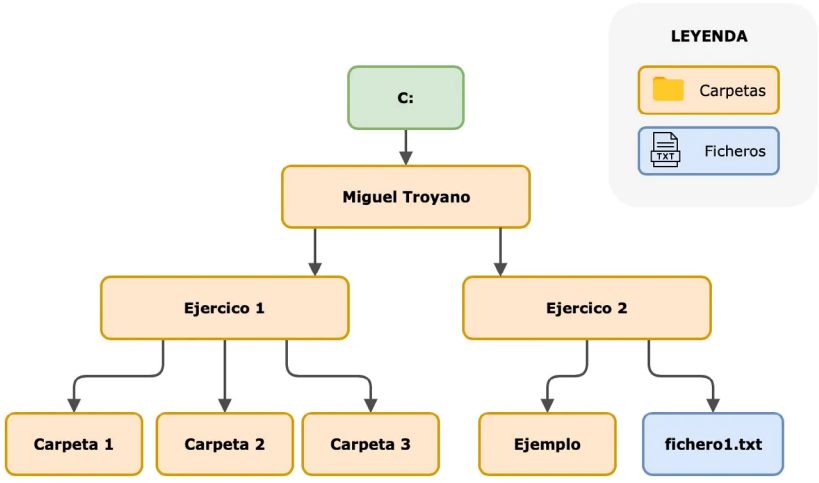


Fig.15. Jerarquía de ficheros y directorios.

Sea cual sea el sistema operativo que usemos, Windows, GNU/Linux o MacOS, los ficheros van a ser accesibles mediante su ruta.

En informática llamamos ruta a la forma que tenemos de **referenciar un archivo dentro de un sistema de archivos**. La ruta va a señalar la posición exacta de nuestro fichero mediante una cadena de caracteres.

Existen **dos tipos** de rutas en informática:

- **Rutas relativas**: Indica la posición del fichero desde la posición donde nos encontremos en el sistema de directorios.
- **Rutas absolutas**: Indica la posición del fichero desde el directorio raíz.

Por ejemplo, en la figura anterior, podemos indicar la ruta al fichero 'fichero1.txt' de las dos formas:

- **Ruta relativa**: Si nos encontramos en la carpeta Data, sería Ejercicio2\ fichero1.txt.
- **Ruta absoluta**: C:\MiguelTroyano\Ejercicio2\fichero1.txt.

Para indicar la ruta de los ficheros, podremos hacerlo tanto con la ruta relativa como con la absoluta.

La forma de indicar la ruta va a cambiar según el sistema operativo, y es más recomendable usar la ruta relativa, aunque los separadores van a cambiar también según el sistema operativo.

Para ello, en Java tenemos la posibilidad de hacer que se ponga el separador automáticamente mediante la siguiente instrucción: **File.separator**.

> **¡! Sabías que...**
>
> Usar File.separator en lugar de caracteres de separación de rutas hardcoded permite que el código sea más portátil y funcione correctamente en diferentes sistemas operativos.

/ 12. Errores de ficheros

Cuando trabajamos con ficheros, vamos a tener que lidiar con multitud de **posibles fallos**. Entre los más comunes nos podemos encontrar:

- El fichero al que queremos acceder no existe.
- El fichero al que queremos acceder está corrompido.
- No se tienen permisos de lectura sobre el fichero.
- No se tienen permisos de escritura sobre el fichero.

Las **excepciones** que nos vamos a encontrar cuando trabajamos con ficheros son:

- **IOException**: Es una excepción común que puede ocurrir al trabajar con ficheros en Java. Esta excepción se lanza cuando ocurre un error durante una operación de entrada/salida (E/S) en el sistema de archivos. Puede ser lanzada por diversas clases y métodos que realizan operaciones de E/S, como la lectura o escritura de archivos. Algunas situaciones que pueden provocar una IOException al trabajar con ficheros incluyen:
  - » Intentar abrir un archivo que no existe.
  - » Intentar acceder a un archivo para el cual no se tienen los permisos adecuados.
  - » Interrupción de una operación de E/S debido a un error de hardware o software.
  - » Error al leer o escribir datos en un archivo debido a problemas en el sistema de archivos o en el dispositivo de almacenamiento.
- **FileNotFoundException**: Es una subclase de IOException en Java y se lanza cuando se intenta acceder a un archivo que no existe en la ruta especificada durante una operación de entrada/salida (E/S). Esta excepción es muy común al trabajar con archivos, especialmente al intentar abrir un archivo para lectura o escritura. Puede ocurrir por diversas razones, como proporcionar una ruta incorrecta al archivo, intentar acceder a un archivo que ha sido eliminado o moverlo desde la última vez que se accedió, o si el archivo está en uso por otro proceso y no se puede abrir.
- **SecurityException**: Es una excepción en Java que se lanza cuando ocurre un problema relacionado con la seguridad durante una operación en tiempo de ejecución. En el contexto de ficheros en Java, esta excepción puede ser lanzada en situaciones donde el acceso a un archivo o recurso del sistema de archivos viola las políticas de seguridad establecidas.
- **NullPointerException**: Es una excepción en Java que se lanza cuando se intenta acceder a un fichero que no ha sido inicializado, es decir, cuando se intenta realizar una operación en un objeto que es null.

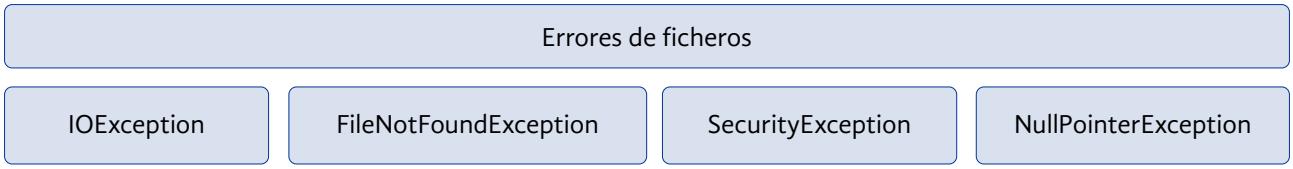


Fig.16. Excepciones de ficheros.

/ 13. Resumen y resolución del caso práctico de la unidad

A lo largo de esta unidad, hemos aprendido el concepto de flujo, qué nos permite hacer y los diferentes tipos de flujos que existen.

Otro concepto que hemos estudiado ha sido el de persistencia de datos, que va íntimamente ligado al de flujo.

Hemos visto cuáles son los flujos de datos predeterminados, aunque los hayamos estado usando sin saberlo desde la primera unidad, y también que existen ficheros tanto basados en bytes como en caracteres.

También se han expuesto las diferentes formas de acceso que tenemos para trabajar con ficheros, analizando sus ventajas e inconvenientes, así como las clases que vamos a necesitar en Java para poder tener acceso a los ficheros y poder operar con ellos de forma cómoda.

Finalmente, se han descrito los dos tipos de rutas que existen y las excepciones que nos vamos a encontrar cuando trabajemos con ficheros.

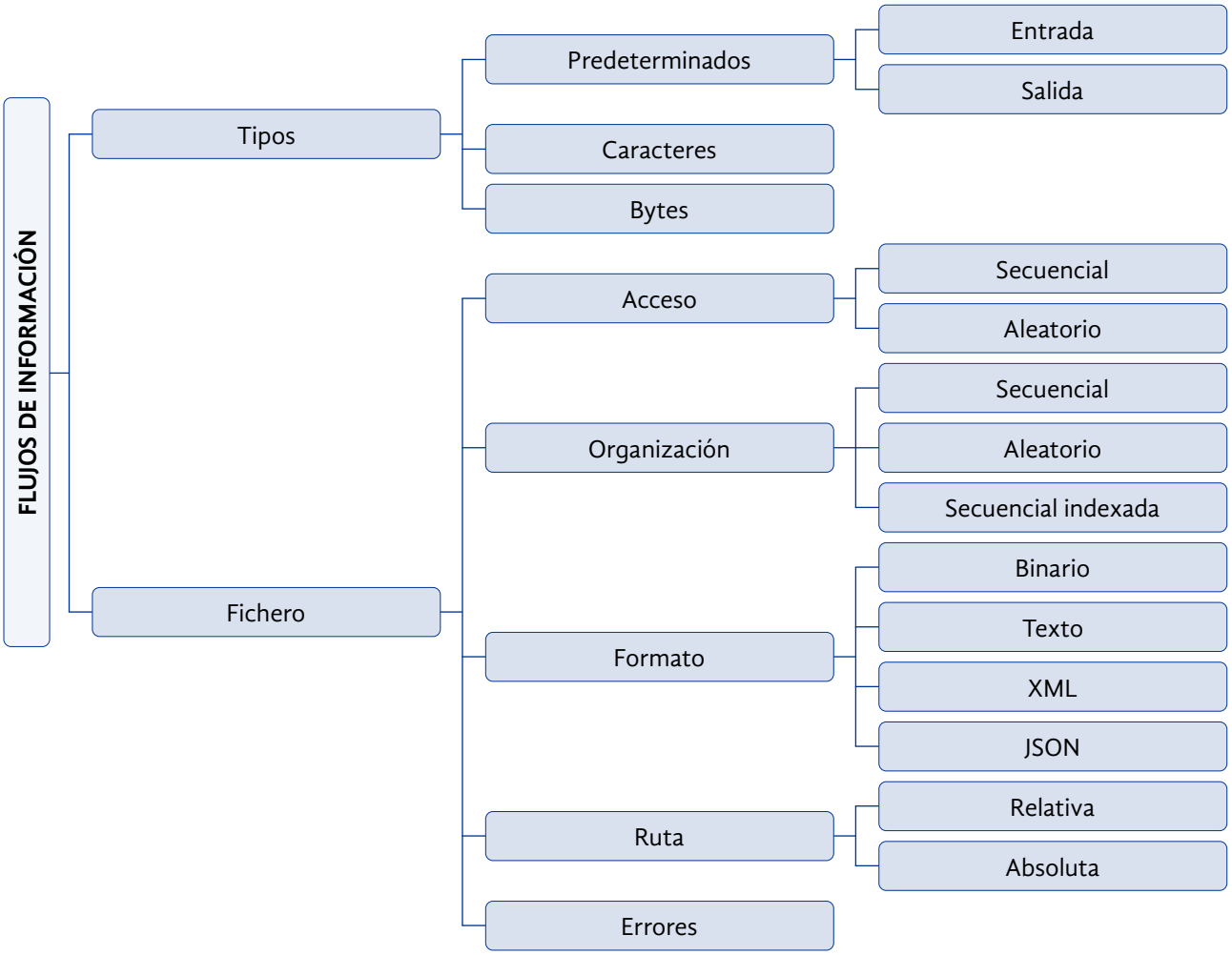


Fig.17. Esquema resumen del tema.

Resolución del caso práctico de la unidad

Uno de los mayores problemas que tenemos hasta el momento es el de que todos los datos que introducimos en nuestros programas se encuentran únicamente en memoria, de esta forma, mientras el programa está ejecutándose no hay ningún problema, pero cuando este se cierra, ya sea por un fallo, por una equivocación o porque necesitemos cerrarlo, todos esos datos que había en memoria se pierden y no habrá forma de recuperarlos.

Este es el gran problema que nos solucionan los ficheros, ya que nos proporcionan persistencia de datos, o lo que es lo mismo, hacen que podamos almacenar información en disco para posteriormente recuperarla y seguir trabajando con ella.

Bien es cierto que los ficheros no son lo más eficiente que podamos tener, hemos visto sus limitaciones con respecto a la eficiencia en los tiempos de acceso, pero eso lo arreglaremos en futuras unidades.

/ 14. Bibliografía

Bloch, J. (2018). *Effective Java (3rd ed.)*. Addison-Wesley.

Caro, C. M.; Ramos, A. N., & Barceló, A. V. (2002). *Introducción a la programación con orientación a objetos*. PRENTICE HALL.

Flujos de datos (s. f.). Recuperado de: http://www.sc.ehu.es/sbweb/fisica/cursoJava/fundamentos/archivos/flujos.htm

Knuth, D. E. (1997). *The Art of Computer Programming (Vol. 1: Fundamental Algorithms) (3rd ed.)*. Addison-Wesley.

Manuel, P. G. J. (2022). *Entornos de desarrollo*. Ediciones Paraninfo, S. A.

Ojeda, F. C. (2001). *Introducción a la programación*. Anaya Multimedia.

Schildt, H. (2018). *Java: The complete reference (11th ed.)*. McGraw-Hill Education.